



CS 498: MoE Efficiency

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

Efficient Architecture

- Mixture-of-Experts (MoE)

Objective:

- Understand the motivation and core ideas behind Mixture-of-Experts

- Compute is the primary challenge of training massive models
- Ambitious model at scale and time to train

Model	Model Size	Hardware	Days to Train
BLOOM	176B	384 A100 GPUs	115 days
OPT	175B	992 A100 GPU	56 days
MT-NLG	530B	2200 A100 GPU	60 days
PaLM	540B	6144 TPU v4	57 days

Next jump in scale:

- Next-generation hardware
- Significant investment in GPUs

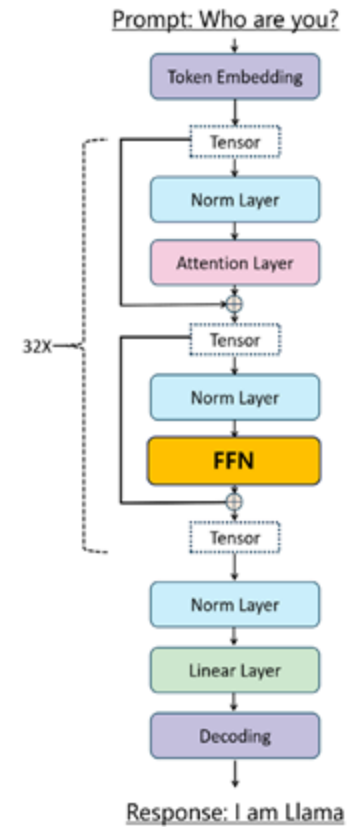
- Tasks/tokens often have different difficulties in inference
- From a computation perspective sparse Mixture-of-Experts provides a promising path
 - Scale at sub-linear cost

Recap: MoE Models are Sparse and Need Less Compute



Dense Models:

- All parameters are used in forward and backward paths
- Increasing model capacity needs more computation
- **Larger model size → Higher compute requirements (FLOPs)**



Recap: MoE Models are Sparse and Need Less Compute

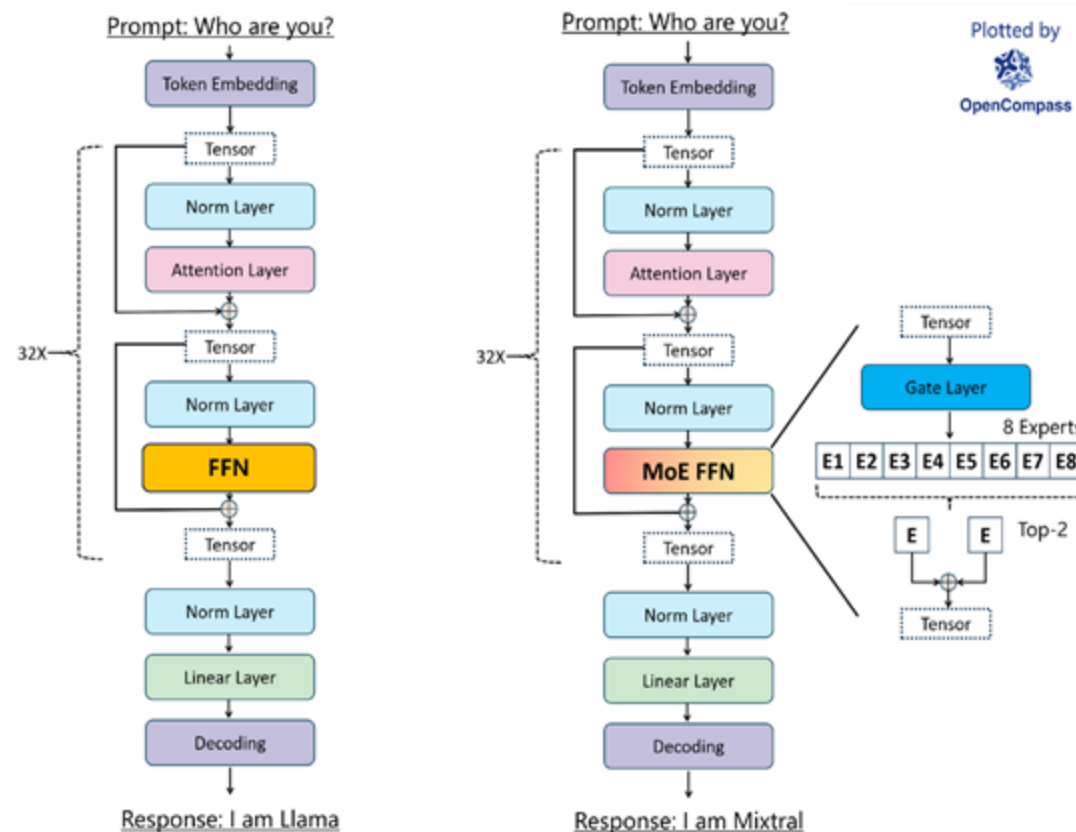


Dense Models:

- All parameters are used in forward and backward paths
- Increasing model capacity needs more computation
- **Larger model size → Higher compute requirements (FLOPs)**

Sparse MoE models

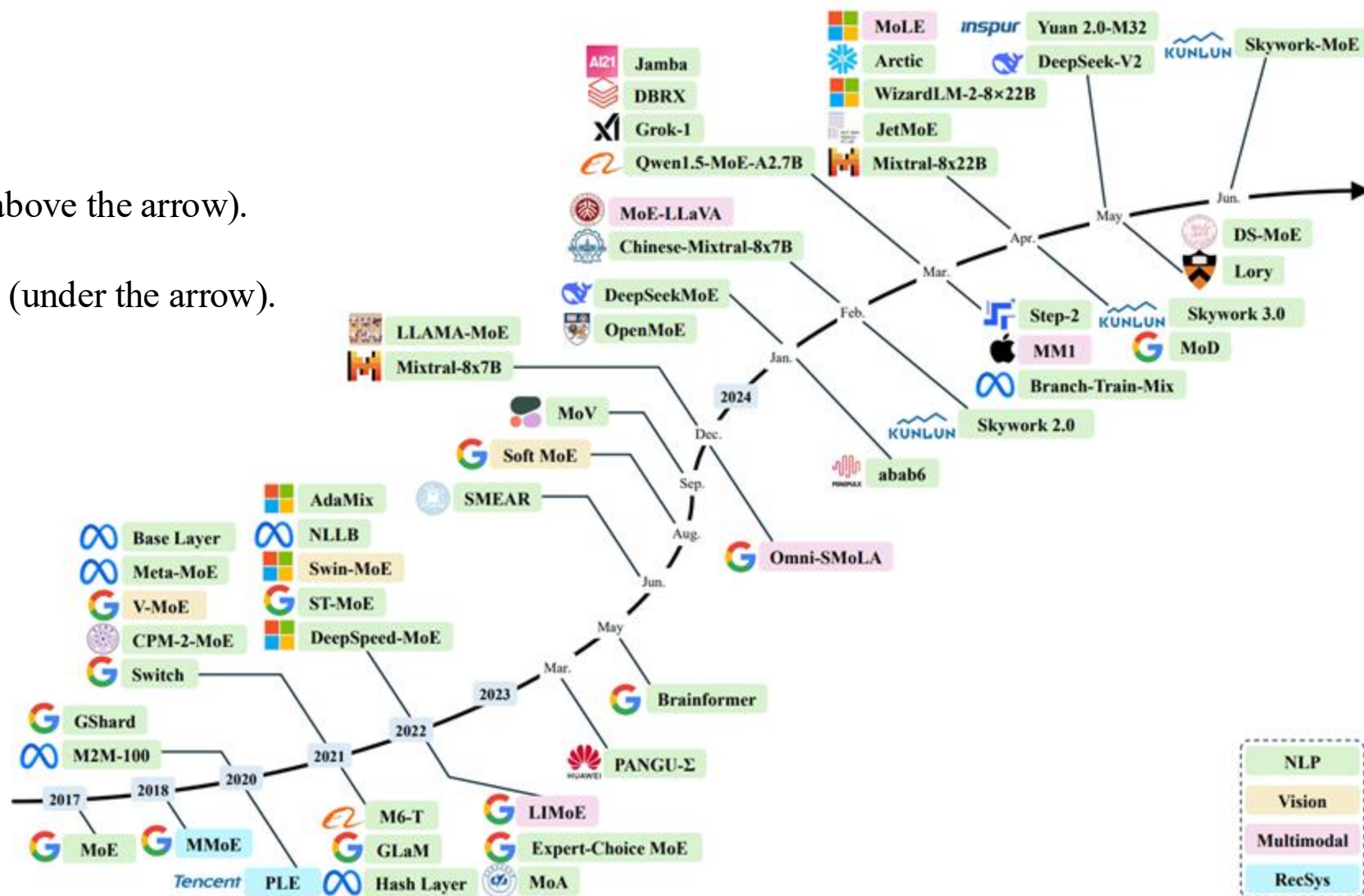
- Sparse utilization of subset of parameters based on input
- Same computation is needed regardless of the model size
- **Larger model size → Similar/Same Compute requirements**



MoE: Road Map



- Open-source (above the arrow).
- Private models (under the arrow).



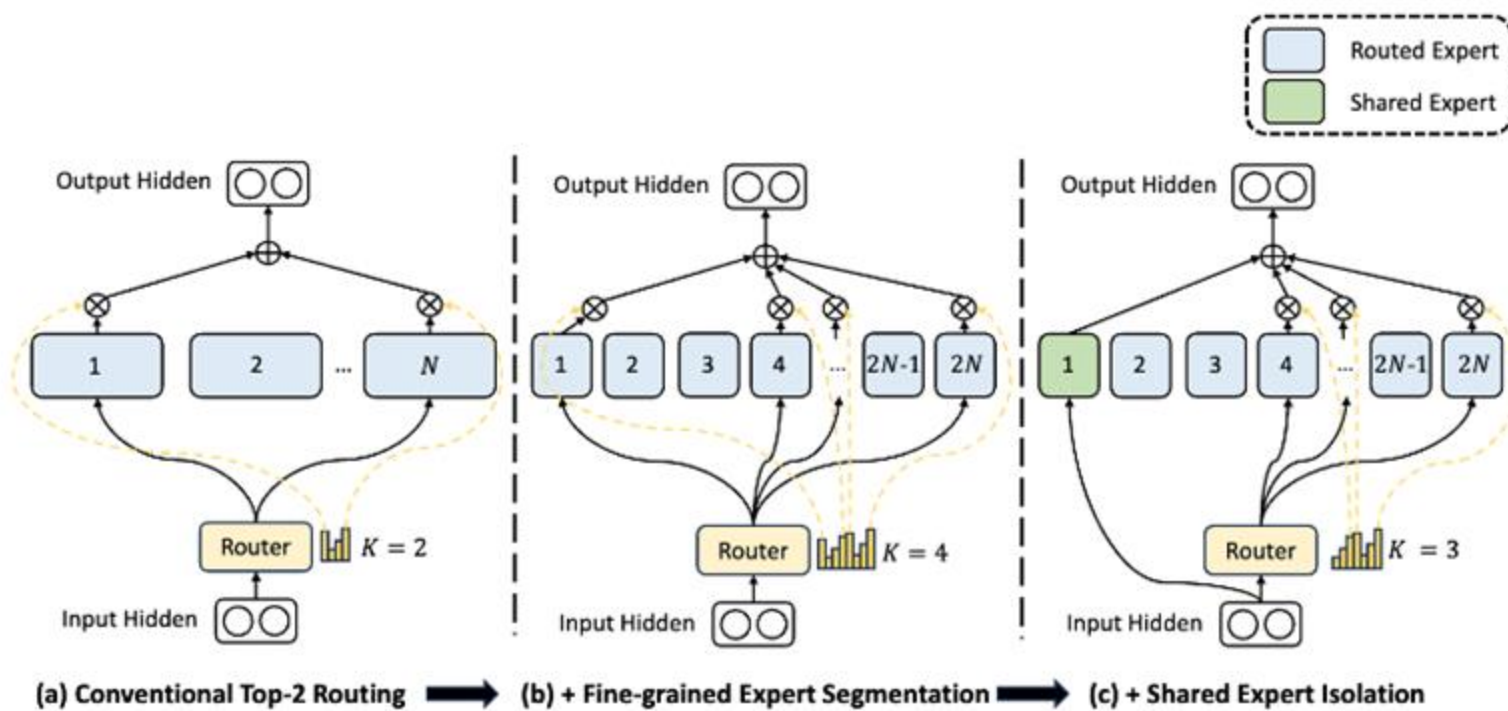
What should we care when designing a MoE?

Network types	FFN, Attention
Fine-grained experts	32 experts/128 experts/...
Shared experts	Isolated experts

What should we care when designing a MoE?

Network types	FFN, Attention
Fine-grained experts	32 experts/128 experts/...
Shared experts	Isolated experts
Activation Function	ReLU/GEGLU/SwiGLU
MoE frequency	Every two layer/Each layer/...
Training auxiliary loss	Auxiliary loss/Z-loss/...

Fine-Grained and Shared Experts



Decompose each expert into sub-experts

Pyramid Design of Experts



- Utilizes more experts in the last few layers as compared to previous layers
- Positive results compared with the baseline MoE

Pyramid Design of Experts



- Utilizes more experts in the last few layers as compared to previous layers
- Positive results compared with the baseline MoE

Model (num. params)	LAMBADA	PIQA	BoolQ	RACE-h	TriviaQA	WebQs
350M+MoE-128 (13B)	62.70	74.59	60.46	35.60	16.58	5.17
350M+PR-MoE-32/64 (4B)	63.65	73.99	59.88	35.69	16.30	4.73
1.3B+MoE-128 (52B)	69.84	76.71	64.92	38.09	31.29	7.19
1.3B+PR-MoE-64/128 (31B)	70.60	77.75	67.16	38.09	28.86	7.73

MoE Expert Design



Reference	Models	Expert Count (Activ./Total)
GShard [86] (2020)	600B	2/2048
	200B	2/2048
	150B	2/512
	37B	2/128
Switch [49] (2021)	7B	1/128
	26B	1/128
	395B	1/64
	1571B	1/2048
GLaM [44] (2021)	0.1B/1.9B	2/64
	1.7B/27B	2/64
	8B/143B	2/64
	64B/1.2T	2/64
DeepSpeed-MoE [121] (2022)	350M/13B	2/128
	1.3B/52B	2/128
	PR-350M/4B	2/32-2/64
	PR-1.3B/31B	2/64-2/128
ST-MoE [197] (2022)	0.8B/4.1B	2/32
	32B/269B	2/64
Mixtral [74] (2023)	13B/47B	2/8
	39B/141B	2/8
LLAMA-MoE [149] (2023)	3.0B/6.7B	2/16
	3.5B/6.7B	4/16
	3.5B/6.7B	2/8
DeepSeekMoE [30] (2024)	0.24B/1.89B	8/64
	2.8B/16.4B	8/66
	22B/145B	16/132
OpenMoE [172] (2024)	339M/650M	2/16
	2.6B/8.7B	2/32
	6.8B/34B	2/32
Qwen1.5-MoE [151] (2024)	2.7B/14.3B	8/64
DBRX [34] (2024)	36B/132B	4/16
Jamba [94] (2024)	12B/52B	2/16
Skywork-MoE [154] (2024)	22B/146B	2/16
Yuan 2.0-M32 [166] (2024)	3.7B/40B	2/32

MoE Expert Design



Reference	Models	Expert Count (Activ./Total)	d_{model}	d_{ffn}	d_{expert}	#L	#H	d_{head}	Placement Frequency	Activation Function	Share Expert Count
GShard [86] (2020)	600B	2/2048	1024	8192	d_{ffn}	36	16	128	1/2	ReLU	0
	200B	2/2048	1024	8192	d_{ffn}	12	16	128	1/2	ReLU	0
	150B	2/512	1024	8192	d_{ffn}	36	16	128	1/2	ReLU	0
	37B	2/128	1024	8192	d_{ffn}	36	16	128	1/2	ReLU	0
Switch [49] (2021)	7B	1/128	768	2048	d_{ffn}	12	12	64	1/2	GEGLU	0
	26B	1/128	1024	2816	d_{ffn}	24	16	64	1/2	GEGLU	0
	395B	1/64	4096	10240	d_{ffn}	24	64	64	1/2	GEGLU	0
	1571B	1/2048	2080	6144	d_{ffn}	15	32	64	1	ReLU	0
GLaM [44] (2021)	0.1B/1.9B	2/64	768	3072	d_{ffn}	12	12	64	1/2	GEGLU	0
	1.7B/27B	2/64	2048	8192	d_{ffn}	24	16	128	1/2	GEGLU	0
	8B/143B	2/64	4096	16384	d_{ffn}	32	32	128	1/2	GEGLU	0
	64B/1.2T	2/64	8192	32768	d_{ffn}	64	128	128	1/2	GEGLU	0
DeepSpeed-MoE [121] (2022)	350M/13B	2/128	1024	$4d_{model}$	d_{ffn}	24	16	64	1/2	GeLU	0
	1.3B/52B	2/128	2048	$4d_{model}$	d_{ffn}	24	16	128	1/2	GeLU	0
	PR-350M/4B	2/32-2/64	1024	$4d_{model}$	d_{ffn}	24	16	64	1/2, 10L-32E, 2L-64E	GeLU	1
	PR-1.3B/31B	2/64-2/128	2048	$4d_{model}$	d_{ffn}	24	16	128	1/2, 10L-64E, 2L-128E	GeLU	1
ST-MoE [197] (2022)	0.8B/4.1B	2/32	1024	2816	d_{ffn}	27	16	64	1/4, add extra FFN	GEGLU	0
	32B/269B	2/64	5120	20480	d_{ffn}	27	64	128	1/4, add extra FFN	GEGLU	0
Mixtral [74] (2023)	13B/47B	2/8	4096	14336	d_{ffn}	32	32	128	1	SwiGLU	0
	39B/141B	2/8	6144	16384	d_{ffn}	56	48	128	1	SwiGLU	0
LLaMA-MoE [149] (2023)	3.0B/6.7B	2/16	4096	11008	688	32	32	128	1	SwiGLU	0
	3.5B/6.7B	4/16	4096	11008	688	32	32	128	1	SwiGLU	0
	3.5B/6.7B	2/8	4096	11008	1376	32	32	128	1	SwiGLU	0
DeepSeekMoE [30] (2024)	0.24B/1.89B	8/64	1280	-	$\frac{1}{4}d_{ffn}$	9	10	128	1	SwiGLU	1
	2.8B/16.4B	8/66	2048	10944	1408	28	16	128	1, except 1st layer	SwiGLU	2
	22B/145B	16/132	4096	-	$\frac{1}{8}d_{ffn}$	62	32	128	1, except 1st layer	SwiGLU	4
OpenMoE [172] (2024)	339M/650M	2/16	768	3072	d_{ffn}	12	12	64	1/4	SwiGLU	1
	2.6B/8.7B	2/32	2048	8192	d_{ffn}	24	24	128	1/6	SwiGLU	1
	6.8B/34B	2/32	3072	12288	d_{ffn}	32	24	128	1/4	SwiGLU	1
Qwen1.5-MoE [151] (2024)	2.7B/14.3B	8/64	2048	5632	1408	24	16	128	1	SwiGLU	4
DBRX [34] (2024)	36B/132B	4/16	6144	10752	d_{ffn}	40	48	128	1	SwiGLU	0
Jamba [94] (2024)	12B/52B	2/16	4096	14336	d_{ffn}	32	32	128	1/2, 1:7 Attention:Mamba	SwiGLU	0
Skywork-MoE [154] (2024)	22B/146B	2/16	4608	12288	d_{ffn}	52	36	128	1	SwiGLU	0
Yuan 2.0-M32 [166] (2024)	3.7B/40B	2/32	2048	8192	d_{ffn}	24	16	256	1	SwiGLU	0

Key points :

- Most recent models place MoE each layer.
- Some of recent models apply shared experts.

Training with different auxiliary loss:

Reference	Auxiliary Loss	Coefficient
Shazeer et al.[135], V-MoE[128]	$L_{importance} + L_{load}$	$w_{importance} = 0.1, w_{load} = 0.1$
GShard[86], Switch-T[49], GLaM[44], Mixtral-8x7B[74], DBRX[34], Jamba[94], DeepSeekMoE[30], DeepSeek-V2[36], Skywork-MoE[154]	L_{aux}	$w_{aux} = 0.01$
ST-MoE[197], OpenMoE[172], MoA[182], JetMoE [139]	$L_{aux} + L_z$	$w_{aux} = 0.01, w_z = 0.001$
Mod-Squad[21], Moduleformer[140], DS-MoE[117]	L_{MI}	$w_{MI} = 0.001$

Importance loss: encourages all experts to have equal importance

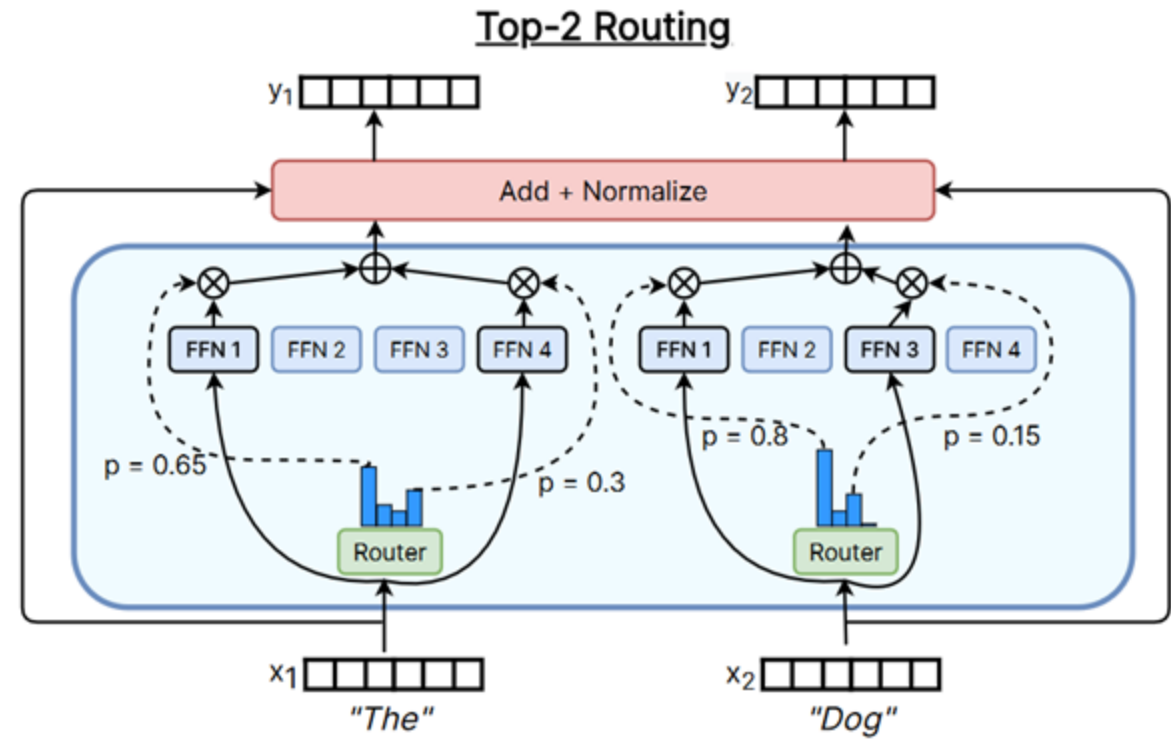
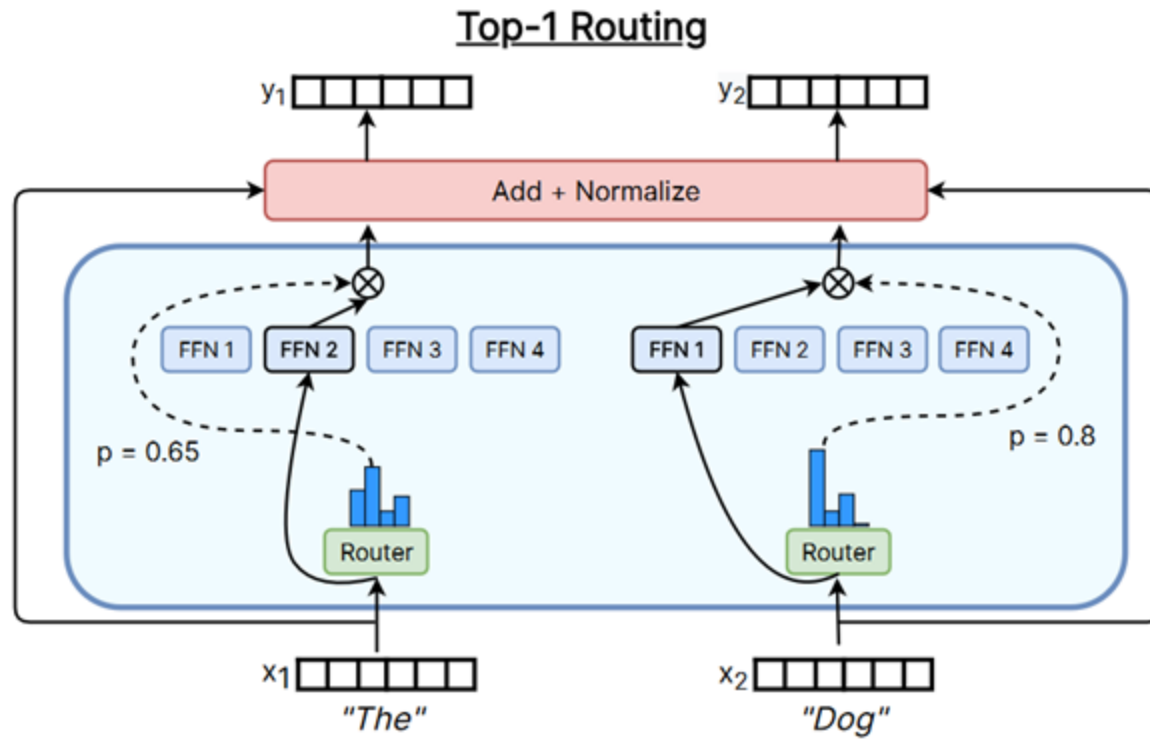
Load loss: ensure balanced loads

Auxiliary loss: mitigating load imbalance losses

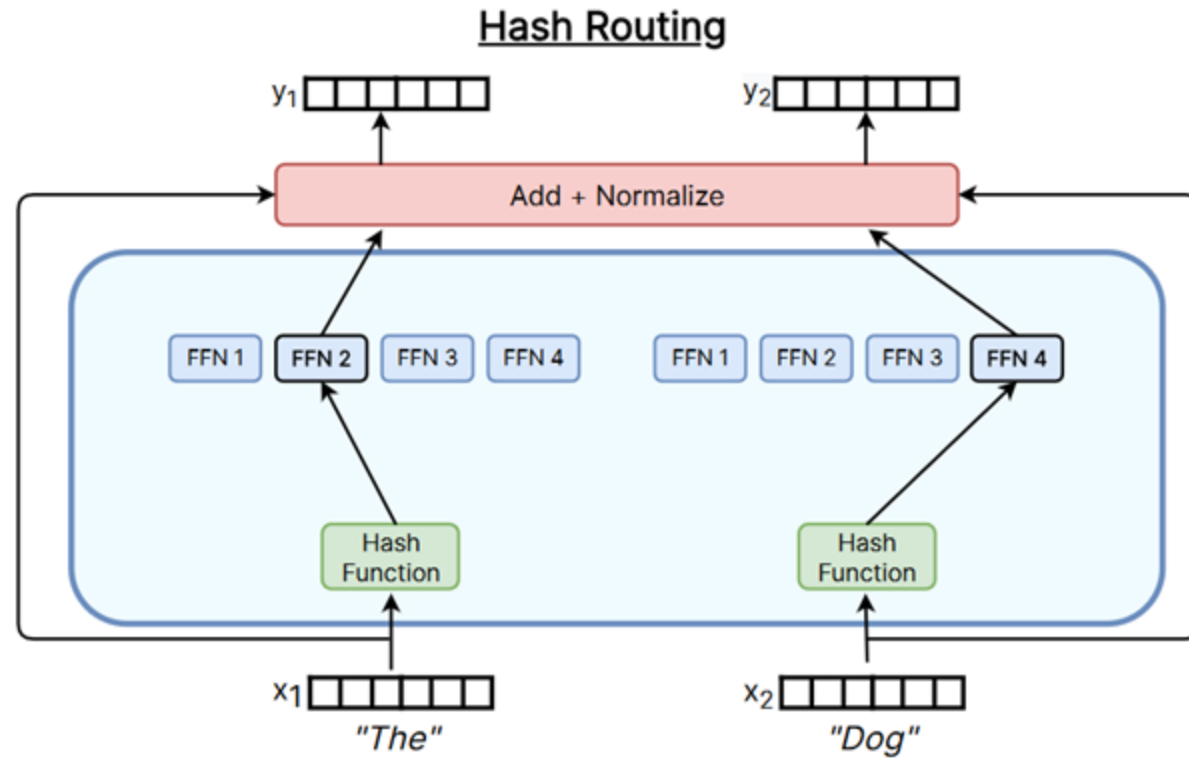
Z-loss: improving training stability by penalizing large logits

MI-loss: mutual information (MI) between experts and tasks to build task-expert alignment

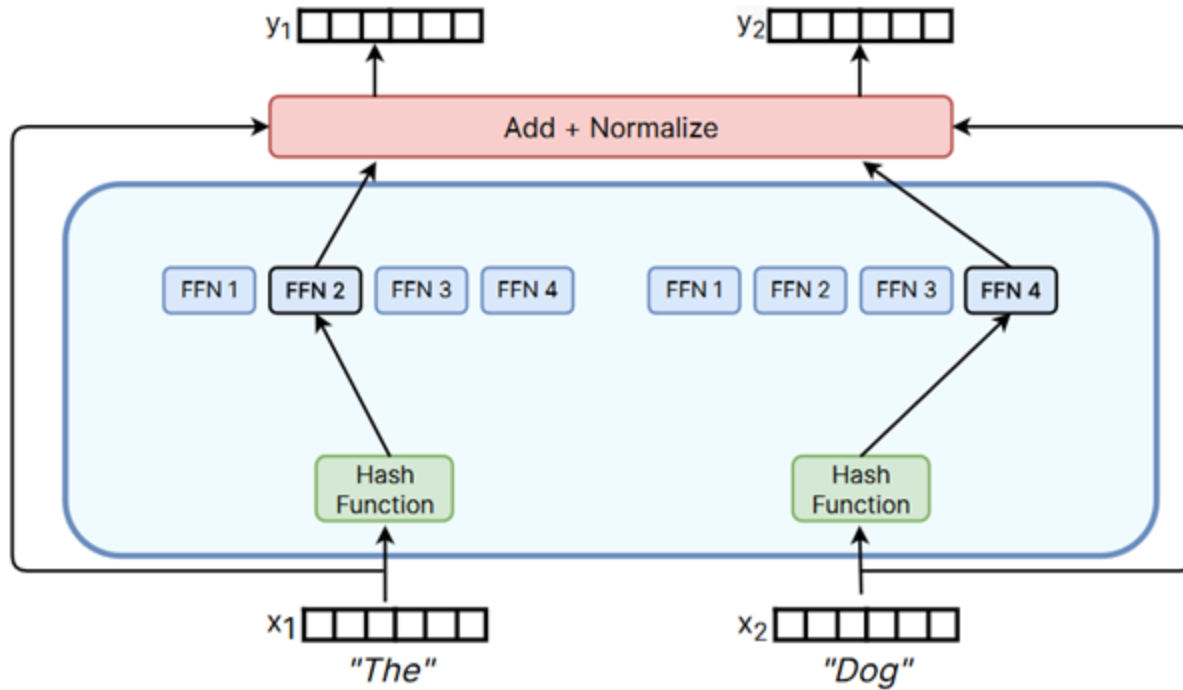
Routing Algorithms



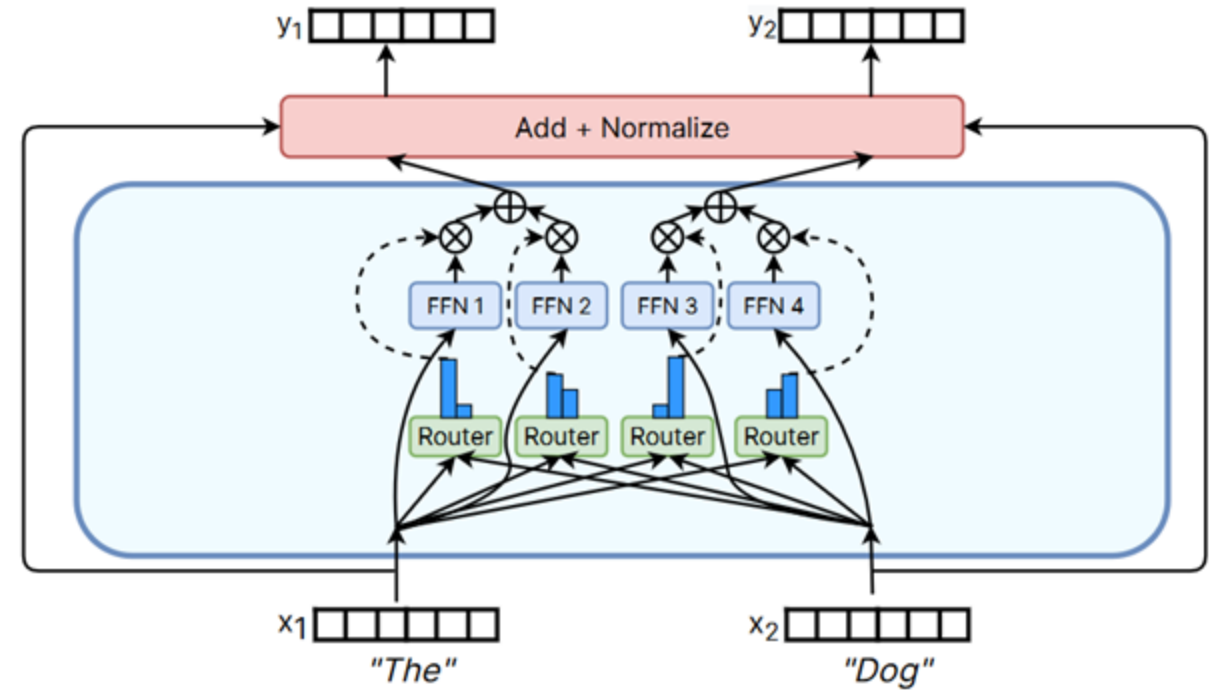
Routing Algorithms



Hash Routing

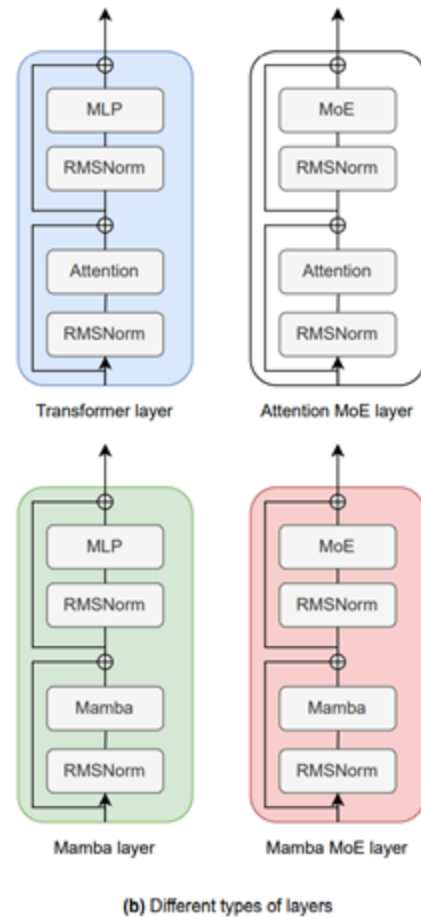


Expert Chooses Tokens



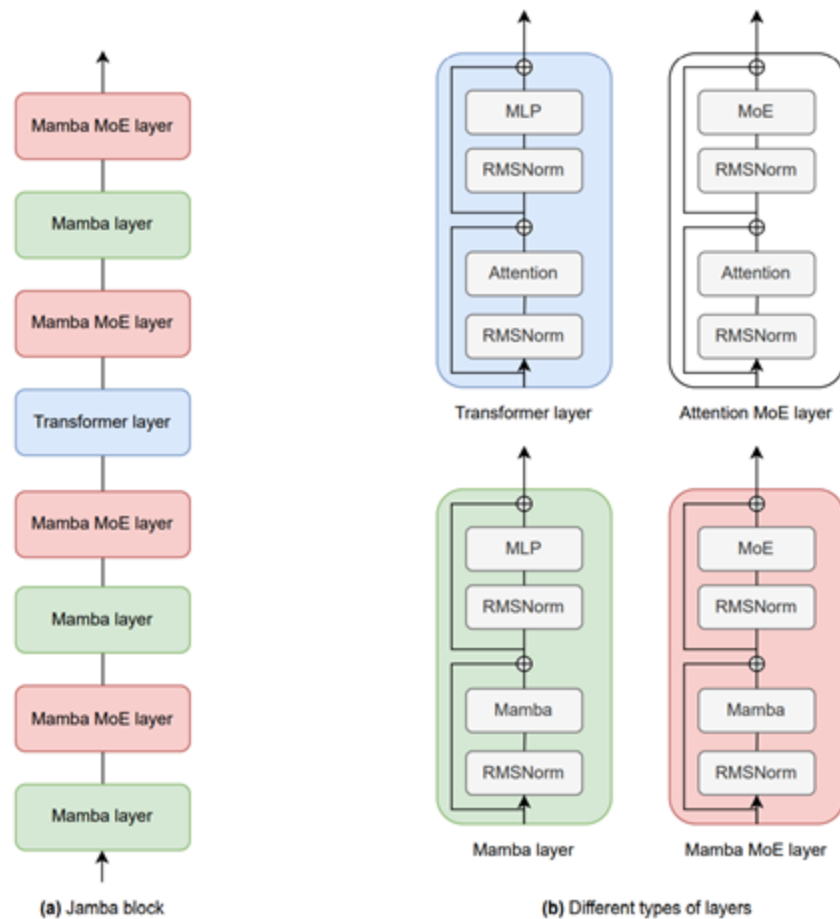
Nice Balance! But may leave out some tokens ...

Example : Jamba (Hybrid architecture)



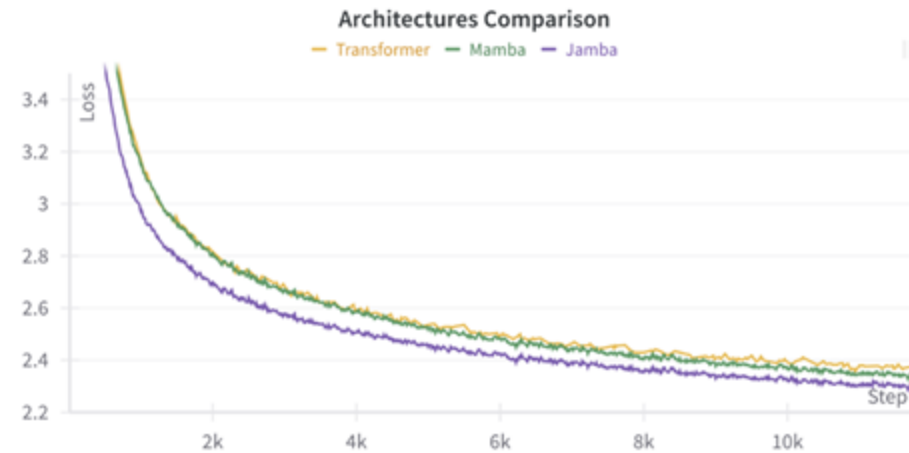
Mamba: A state space model (~RNNs), a counterpart of transformer
- Faster inference (linear scaling in sequence length) and lower memory footprint compared to transformer

Example : Jamba (Hybrid architecture)

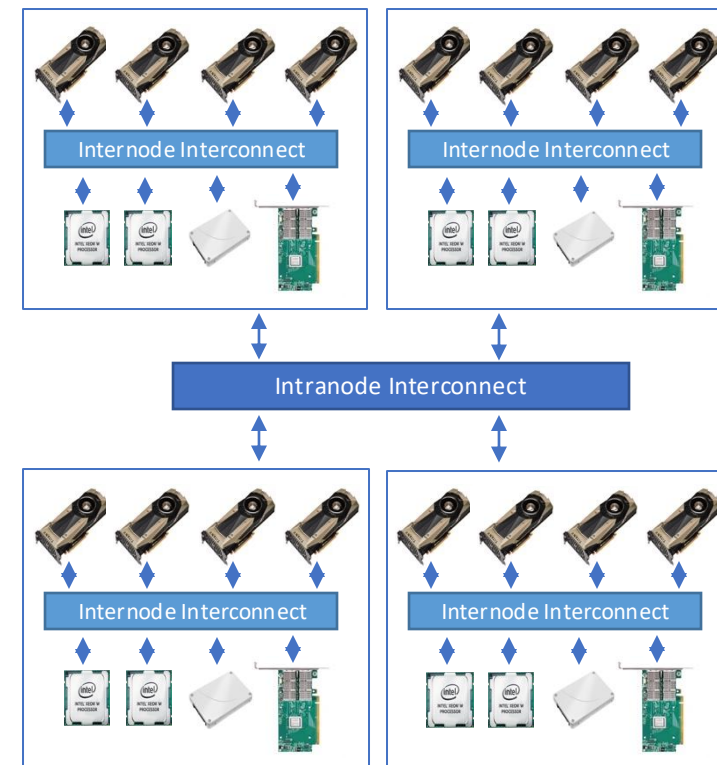
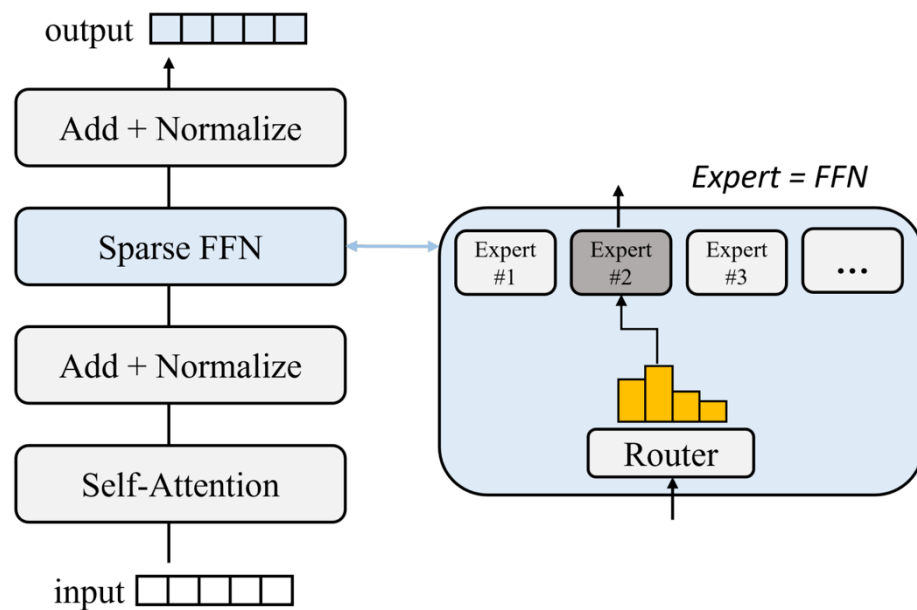


Jamba is a hybrid decoder architecture that mixes Transformer layers with Mamba layers, in addition to a mixture-of-experts (MoE) module.

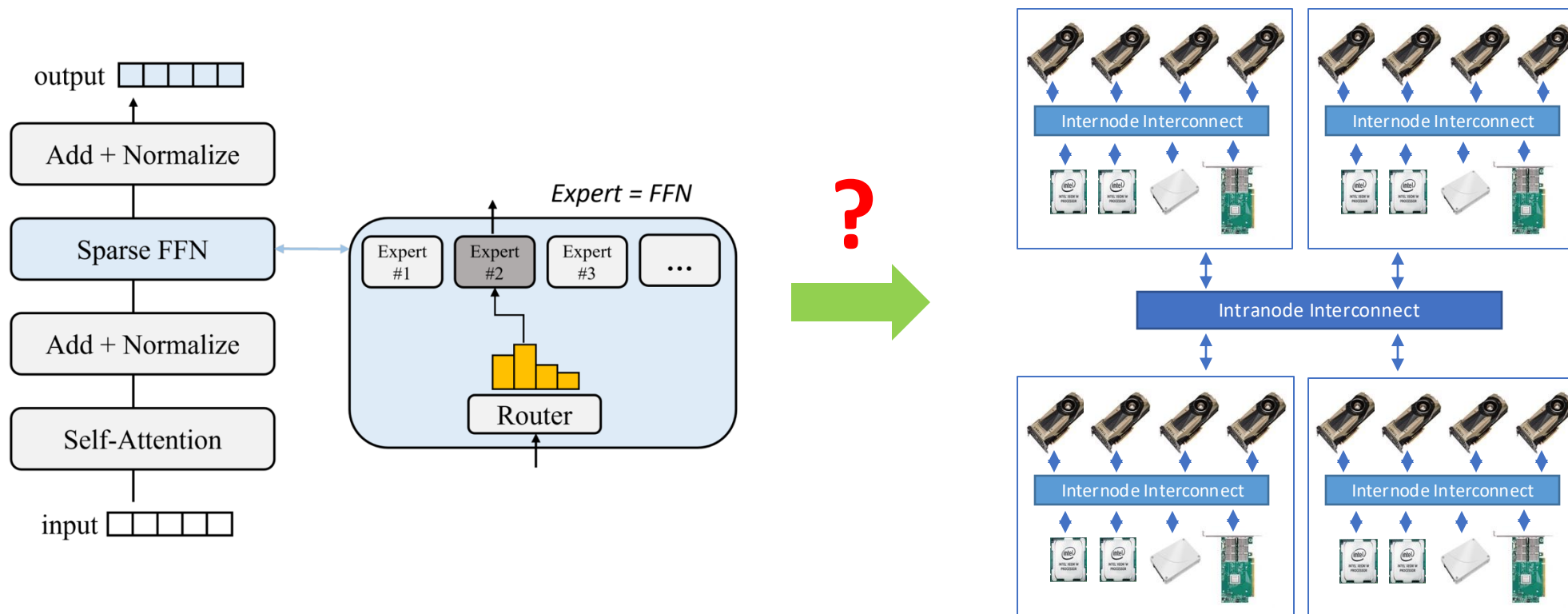
	Available params	Active params
LLAMA-2	6.7B	6.7B
Mistral	7.2B	7.2B
Mixtral	46.7B	12.9B
Jamba	52B	12B



MoE Scaling Challenges on Modern Hardware with Massive Parallelism

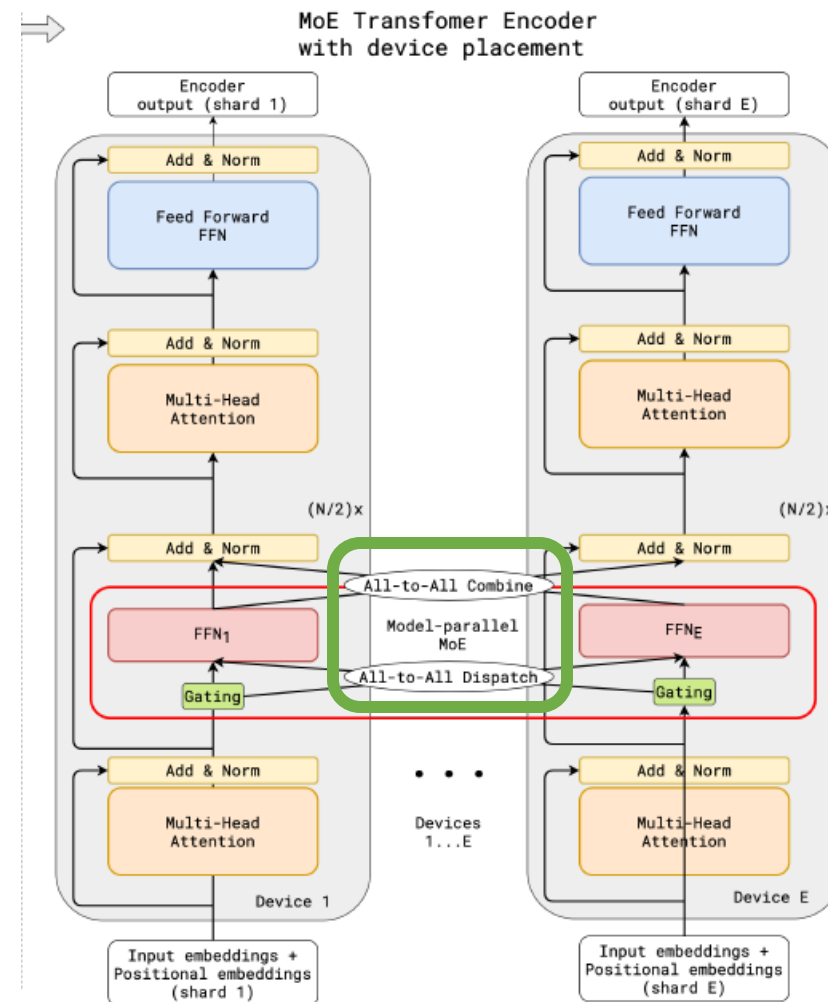


MoE Scaling Challenges on Modern Hardware with Massive Parallelism

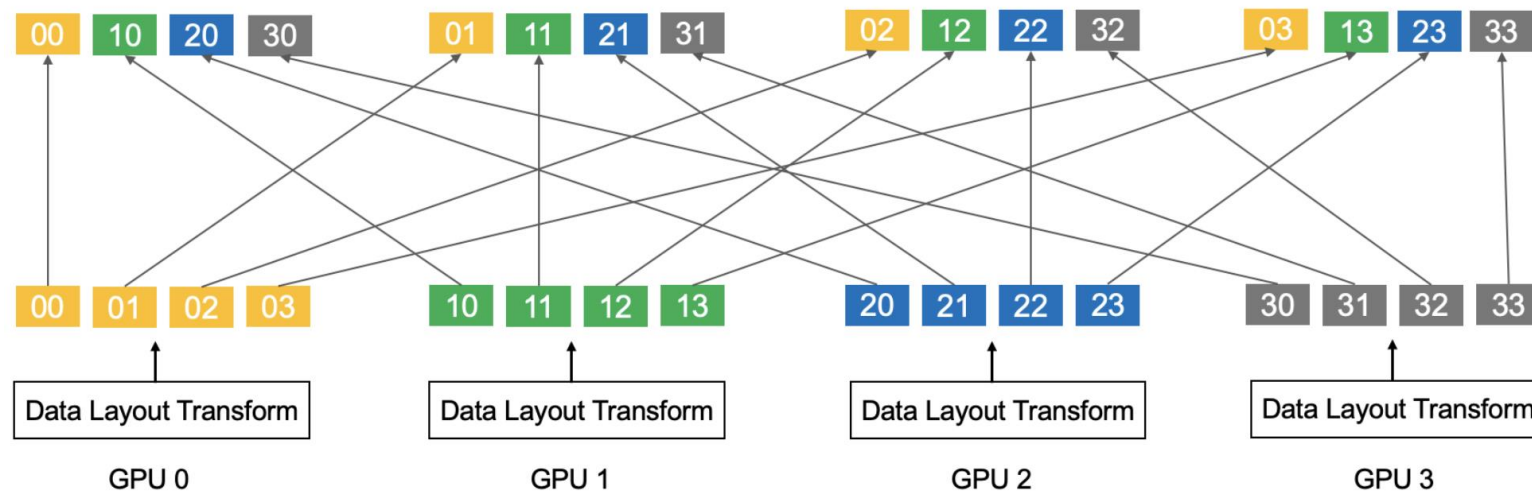


- How to break the memory wall to enable massive MoEs?
- How to efficiently route tokens to different experts across GPUs?
- How to minimize communication overhead while achieving high per-GPU compute throughput?

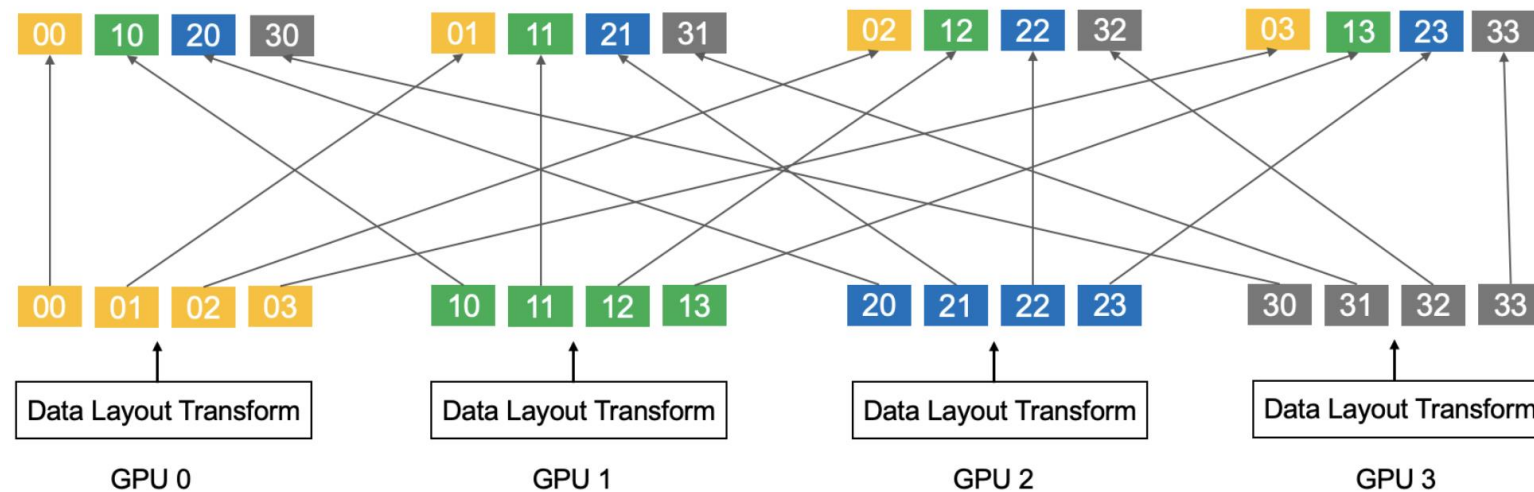
- Expert parameters – partitioned (sharded)
 - Like model parallelism (MP)
 - Each expert process a subset of tokens
- **Two All-to-All(s) in Forward and Backward**



1. Gating function (router): decide target experts for each token
2. **Dispatch phase:**
 - a. 1st layout transformation: tokens to the same target experts are grouped in a continuous memory buffer
 - b. 1st All2All: dispatch tokens to their corresponding experts



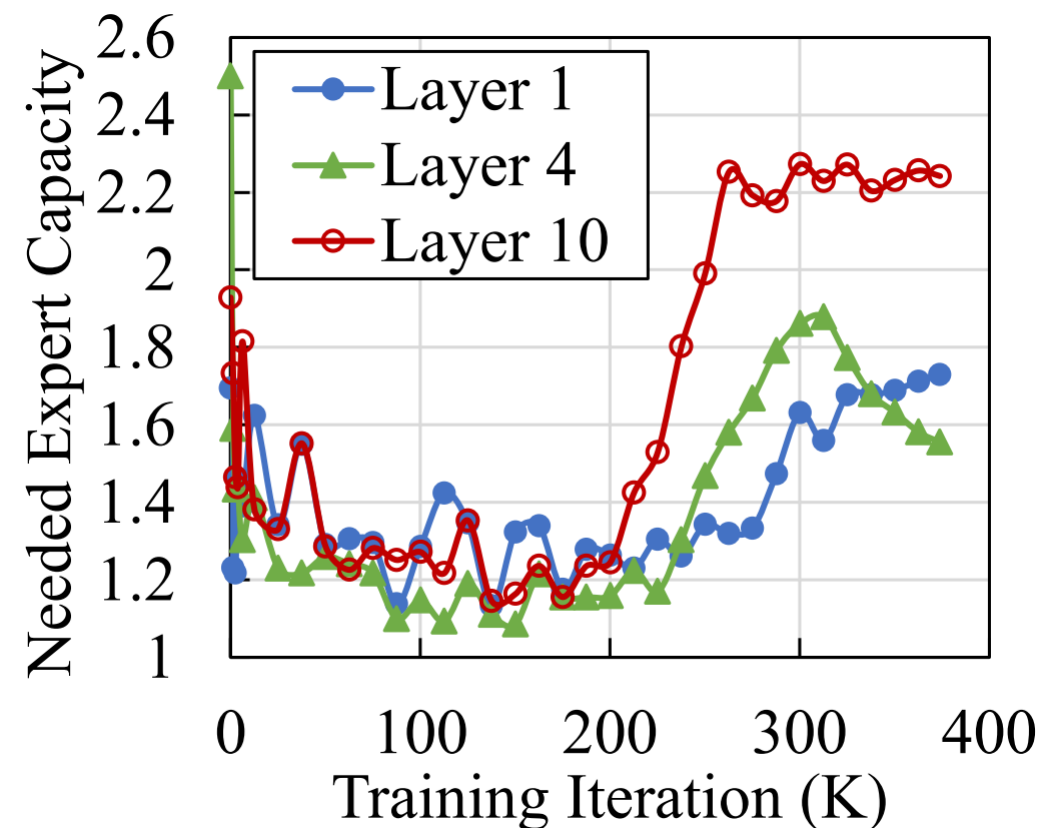
1. Gating function (router): decide target experts for each token
2. **Dispatch phase:**
 - a. 1st layout transformation: tokens to the same target experts are grouped in a continuous memory buffer
 - b. 1st All2All: dispatch tokens to their corresponding experts
3. Expert compute: each expert process its tokens
4. **Combine phase:**
 - a. 2nd All2All: combine processed tokens back to their GPUs
 - b. 2nd layout transformation: restore tokens to their original positions



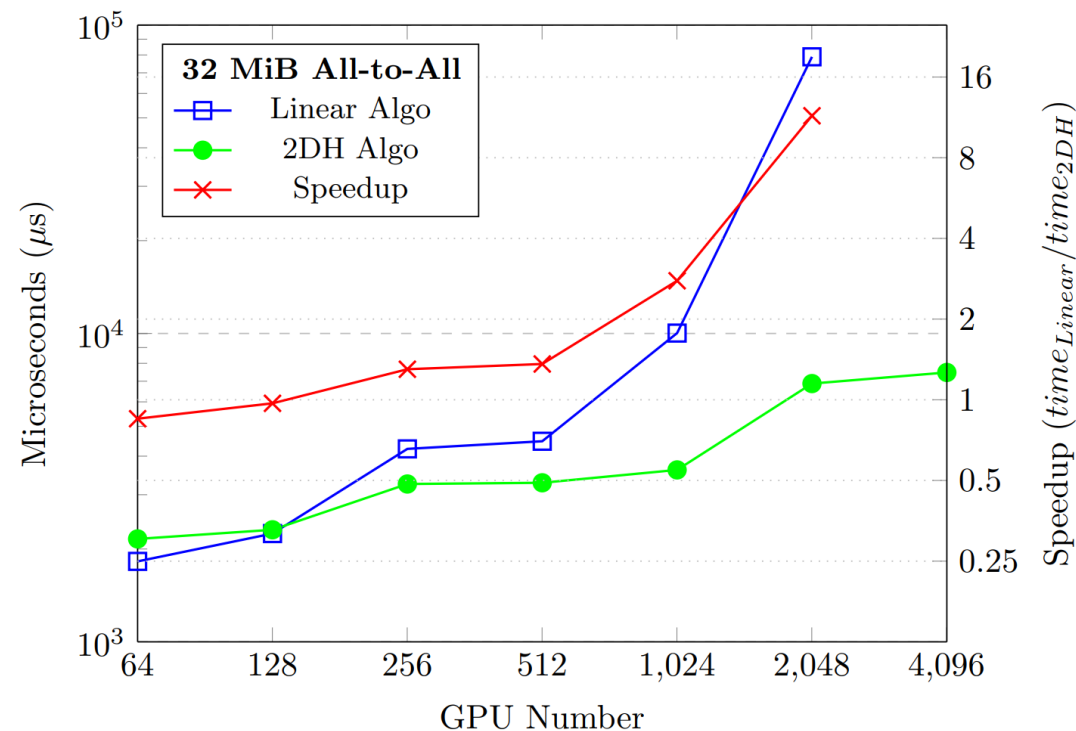
Short Name	Flexible Parallelism Combinations	Benefit
E	Expert	Scales the model size by increasing the number of experts
E+D	Expert + Data	Accelerates training throughput by scaling to multiple data parallel groups
E+Z	Expert + ZeRO	Partitions the nonexpert parameters to support larger base models
E+D+M	Expert + Data + Model	Supports massive hidden sizes and even larger base models than E+Z
E+D+Z	Expert + Data + ZeRO	
E+Z-Off+M	Expert + ZeRO-Offload + Model	Leverages both GPU and CPU memory for large MoE models on limited GPU resources

Optimal parallelism strategy depends on model and hardware specifics

- Observation: Workload per expert changes during training
- Solution: Dynamically adapt parallelism DP + EP vs. TP + EP

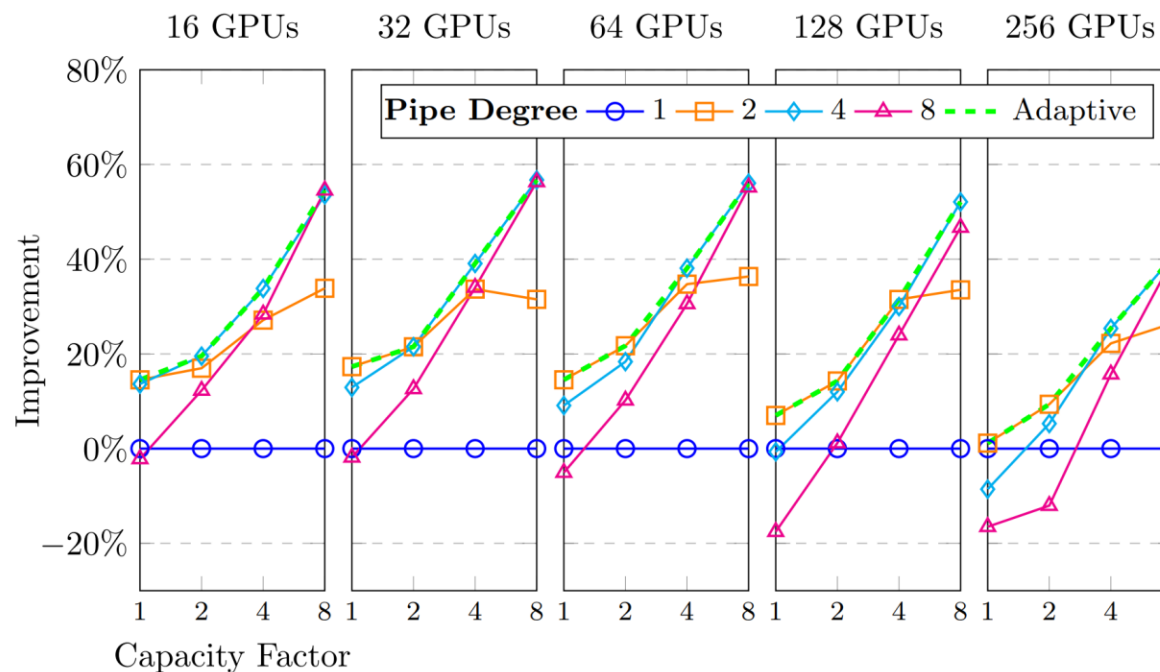


- Observation: All2all is expensive across nodes and with many small messages
- Solution 1: Take into account of network hierarchy with 2D hierarchical all2all: Intra-node all2all + Inter-node all2all
- Solution 2: Leverage highly-optimized communication collectives from MSCCL (Microsoft)



Up to 10x all2all speedup

- Observation: Token partitioning + concurrent CUDA kernels => pipeline parallelism that overlap all2all with FFN layer compute
- Solution: Adaptive pipeline degree based on workloads

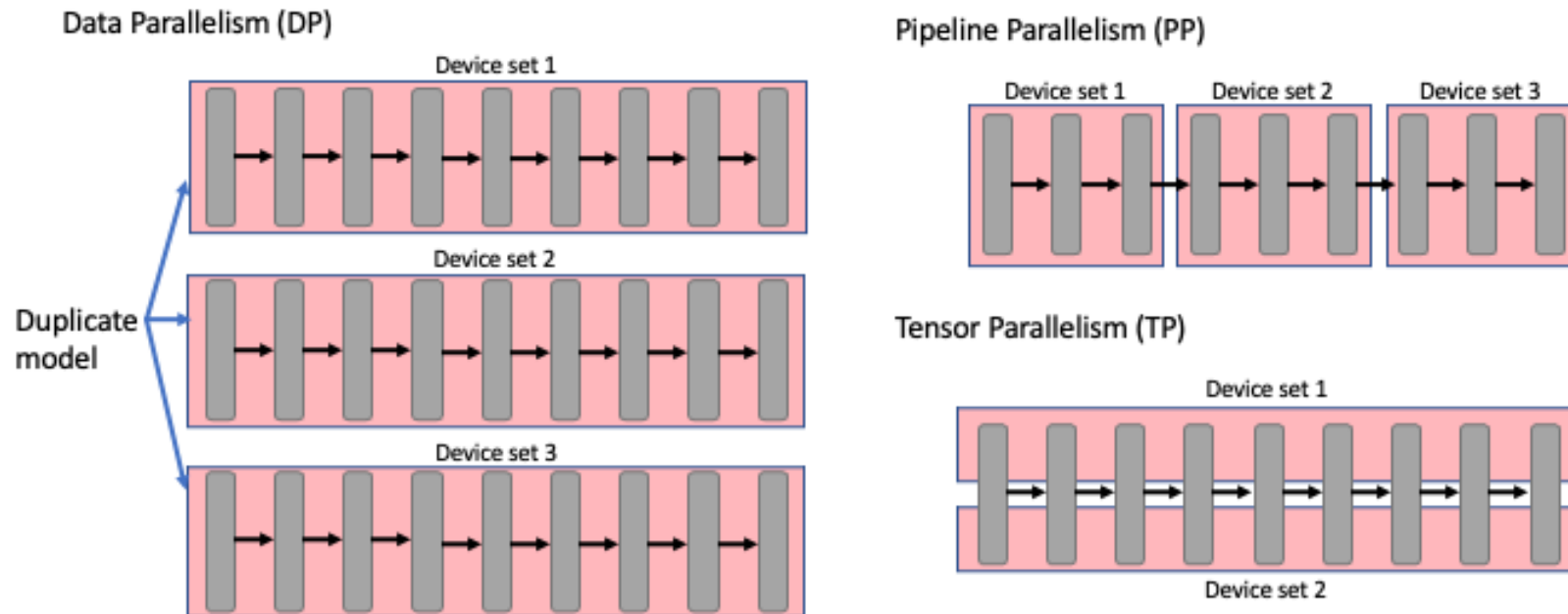


Up to 57% improvement in comparison to pipeline degree 1

- Novel MoE architecture and training objective design
- Expect more optimizations against the training efficiency of MoE models, e.g., parameter-efficient MoE, multi-modal MoE
- Subsequent extensions of MoE based foundation models to diverse use cases
- System optimizations that leverage heterogeneous hardware resource to lower the cost of training and fine-tuning MoE
- Efficient MoE inference systems to achieve low latency and high-throughput

- We have completed the core lecture material (Great job!)
- Will have two lectures covering advanced topics (e.g., RAGs) and one guest lecture on vision systems

- **Model execution exceeds the capacity of single machines**
 - Data parallelism, pipeline parallelism, tensor parallelism

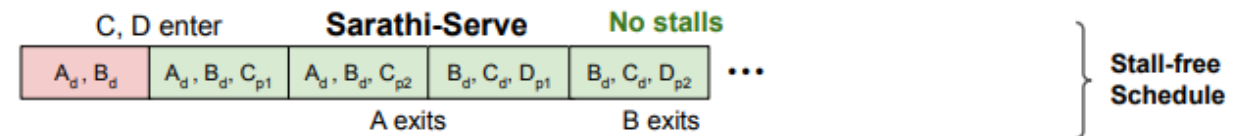
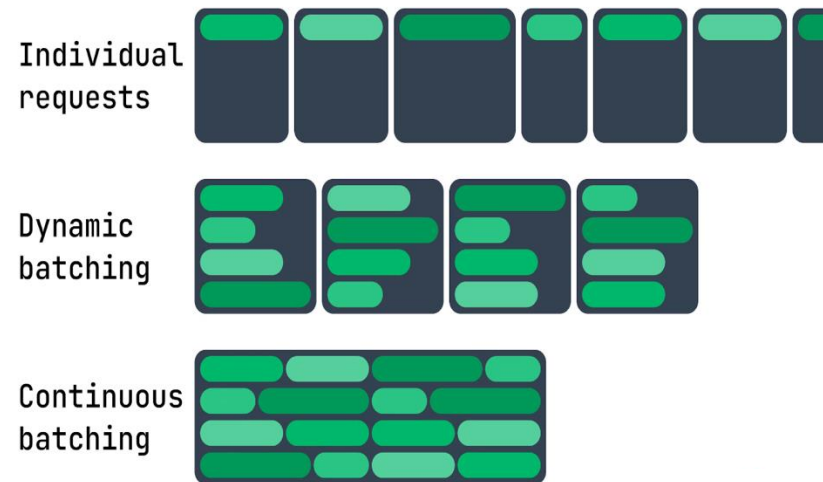


- **Model execution exceeds the capacity of single machines**
 - Data parallelism, pipeline parallelism, tensor parallelism
- **Reduce the resource demand of training**
 - Mixed precision training (e.g., INT8)
 - Gradient accumulation
 - Activation checkpointing
 - Low-rank adaptation

- **FlashAttention**
 - Reduce IO during attention computation via tiling
 - Compute, combine, and fuse kernels

- **Inference Batching**

- Continuous Batching: selectively fill requests after each decoding step
- Chunked-Prefill: mix prefilling and decoding requests for better throughput



- **Inference Batching**

- Continuous Batching: selectively fill requests after each decoding step
- Chunked-Prefill: mix prefilling and decoding requests for better throughput

- **Optimizing KV Cache**

- PagedAttention: allocate on-demand memory in a paging manner to reduce fragmentation
- H2O: retain the tokens with high attention scores to cap memory usage
- AttentionSink: keep the first several tokens and ones in the last window
- Quantization: Per-Token/Tensor/Channel/Group QAT

- **Document Your Contributions in Mid-Semester & Final Reports**

- Individual grades will partially reflect each member's contribution
- Feel TOTALLY free to take up challenging or risky tasks
- Statement template at the end of the report:

Team Member Contributions:

```
\begin{itemize}
```

```
\item \emph{Member's NetID}: Contribution to date; Planned contribution  
toward final report and presentation:
```

```
\item \emph{Member's NetID}: Contribution to date; Planned contribution  
toward final report and presentation:
```

```
\end{itemize}
```

Questions?